

Trabajo Práctico Final

Redes Neuronales aplicadas a Visión

Introducción	1
Cómputo en la Nube	2
Entregable	3
1. Clasificación de Imágenes	7
2. Segmentación de patologías en imágenes médicas	13
3. Monitoreo de vida silvestre utilizando Imágenes RGB y termales	15
4. Super-Resolución de Imágenes	18
5. Colorización de imágenes	23
6. Síntesis de vistas utilizando NeRFs	26
7. UdeSA 3D: Structure-from-Motion	29
8. Propuesta de trabajo por estudiantes	34

Introducción

En el trabajo práctico final deberán abordar una problemática mediante el desarrollo de modelos de aprendizaje profundo basados en conjuntos de datos reales.

El TPF se realizará de manera **en grupos de hasta tres estudiantes**.

La cátedra provee una lista de problemas a abordar de los cuales podrán elegir.

De manera excepcional, la cátedra permitirá que aborden un tema que no sea uno de los propuestos por la cátedra, el cual requerirá la aprobación del Profesor a cargo.

El TPF tendrá una nota numérica del 0 al 10. Esta nota se basará en la evaluación que hagan los docentes del **Informe**, **Código Fuente** y la presentación de un **Poster**.

Se realizará una jornada de presentación de pósters en la universidad.

La cátedra anunciará la fecha por los medios habituales de la cursada.

Cómputo en la Nube

Servicios de cómputo en la nube gratuitos:

→ [Kaggle](#):

- ◆ Acceso a cómputo con GPUs
- ◆ Permite ejecutar jupyter notebooks **en background sin interrupción**
- ◆ Límite de 30hs de cómputo a la semana.
- ◆ [Weekly Maximum GPU Usage](#) / [Efficient GPU Usage Tips and Tricks](#)

→ [Google Colab](#):

- ◆ Acceso a cómputo con GPUs gratuito y sin suscripción
- ◆ **No permite ejecución en background**
- ◆ Límite a las horas de cómputo contiguas
- ◆ Requiere el ingreso de un captcha cada cierta cantidad de minutos

→ [Microsoft Azure Portal](#):

- ◆ Suscripción estudiantil gratuita con 100 usd de crédito
- ◆ [Azure for Students](#)
- ◆ La suscripción gratuita **no permite crear VMs con GPUs**
- ◆ Permite seleccionar la imagen del sistema operativo a virtualizar
- ◆ Ejecución en background sin interrupción (hasta terminar el crédito)

→ [Google Cloud Platform](#):

- ◆ **Requiere información de una tarjeta de crédito para suscribirse**
- ◆ Período de prueba gratuito de 90 días con 300 usd en crédito
- ◆ [Get started for free / 90-day, \\$300 Free Trial offer](#)
- ◆ Acceso a GPUs (las **L4 tienen más disponibilidad que las T4**)

Se recomienda **utilizar Google Colab** (gratuito) **para desarrollar** los modelos, prototipar y **realizar pruebas pequeñas/medianas**. Una vez hayan comprobado que el entorno de entrenamiento y testing esté funcionando correctamente, pueden utilizar servicios de cómputo en la nube de mayor desempeño.

En muchos casos una máquina virtual (VM) con una CPU de alto desempeño en plataformas como Azure y Google Cloud ya proveen una velocidad de entrenamiento aceptable y superior a la de una computadora de escritorio personal.

Se recomienda, en los casos de utilizar VMs en la nube, conectar el IDE VSCode a través de su funcionalidad de [SSH Remote Server](#) para poder inspeccionar directorios, editar archivos, ejecutar comandos y trabajar de forma remota utilizando la interfaz gráfica del VSCode.

Entregable

El entregable debe contener:

- **Informe.**
- **Código fuente** utilizado para resolver el problema
- **Enlaces o referencias a los *datasets* utilizados**
- **Póster**

Si el entregable supera el límite de espacio del Campus Virtual deberán entregar por el campus únicamente el **informe dónde deberá indicar en el texto un enlace a Google Drive** con el entregable completo.

Informe

Deberán presentar un informe de **no más de 8 páginas A4**, siguiendo el formato [IEEE Conference Proceedings](#) exponiendo las técnicas y decisiones tomadas a lo largo del trabajo.

Se recomienda que el informe posea al menos 5 secciones, **introducción, método, conjuntos de datos, resultados y discusión.**

Deberán explicar los métodos utilizados detallando arquitecturas de redes neuronales utilizadas, así como también dar detalles del conjunto de datos sobre el que trabajaron. El nivel de detalle debe ser acorde con la extensión máxima del informe considerando que deben **exponer métricas de precisión e imágenes de los resultados** obtenidos.

Pueden utilizar implementaciones existentes liberadas por investigadores del área. En esos casos, deberán explicar las fuentes, sus detalles y particularidades. En todos los casos deberán proponer al menos una modificación, mejora o tipo de uso ideado por ustedes.

Póster

La diagramación del Poster queda a entera discreción de los estudiantes, pero deberá tener el “Título” y “Autores” del mismo en la parte superior. Se le dará un puntaje al Póster en base a la claridad de la presentación gráfica y oral, y del dominio que demuestren los estudiantes sobre los temas de su TPF.

El póster podrá ser impreso en **tamaño A0 o A1 con orientación vertical**, y la impresión es responsabilidad de los estudiantes. Se recomienda buscar el servicio de impresión con anticipación para evitar contratiempos.

- Pueden crear su póster utilizando herramientas como LaTeX, PowerPoint, Keynote, Canva, Illustrator, entre otras.
- Se recomienda usar un template para facilitar el diseño.
- El póster debe ser de **tamaño A0 o A1 con orientación vertical**.

1. Planificación de Póster

- **Mensaje principal:** Antes de empezar el diseño, definan cuál es el mensaje clave que quieren transmitir.
¿Qué es lo más importante que quieren que la audiencia recuerde?
- **Audiencia:** Consideren a quién va dirigido el póster para ajustar el nivel de detalle y la terminología. **Deberán presentarlo a los profesores de la materia y otros estudiantes de la carrera** interesados.

2. Estructura del Póster

La diagramación del Poster queda a entera discreción de los estudiantes, pero deberá tener el “Título” y “Autores” del mismo en la parte superior.

- **Título**
 - Debe ser claro, conciso y reflejar el contenido del trabajo.
 - Es importante que el título represente la extensión del trabajo, más allá de la tarea básica, evitando títulos genéricos como “Estimación de Precios de Alquiler en AMBA” y enfocándose en las particularidades de su extensión para diferenciarse.
 - Utilicen un tamaño de fuente grande para que se pueda leer desde lejos.
- **Autores**
 - Incluyan sus nombres, la institución a la que pertenecen y sus emails. Es recomendable ponerlo justo debajo del título.

Las diferentes secciones no tienen que llamarse igual que en el informe ni seguir la misma estructura. Estas son sugerencias generales, que pueden adaptarse libremente según las necesidades y características de su trabajo:

- **Metodología / arquitecturas de red**
 - Describan brevemente el enfoque y los métodos utilizados en su investigación.
- **Resultados**
 - Presenten sus hallazgos de manera clara y compacta.
 - Utilicen gráficos, tablas y/o imágenes para facilitar la comprensión.

- **Conclusiones**
 - Resalten los puntos más importantes y las implicaciones de sus resultados.
- **Trabajo a Futuro**
 - Indiquen posibles líneas de investigación o próximos pasos.
 - Si tuvieran otros 6 meses para trabajar en esto, ¿qué harían primero?
- **Referencias**

3. Diseño Visual

- **Colores y Tipografía**
 - Utilicen una paleta de colores coherente y tipografías legibles.
 - Eviten combinaciones de colores que dificulten la lectura.
 - Eviten usar más de 3 colores diferentes. Tengan en cuenta que demasiados colores hacen que el diseño se vea poco profesional, mientras que no tener color lo convierte en algo monótono.
 - El fondo nunca debería distraer del contenido en sí, así que eviten cualquier diseño que sea demasiado cargado.
- **Tamaños de Letras Sugeridos**
 - **Título:** 72-100 pt
 - **Autores:** 40-70 pt
 - **Títulos de Secciones (Ejemplo: Resumen):** 36-48 pt
 - **Texto Principal:** 24-36 pt
 - **Leyendas de Gráficos/Imágenes:** 18-24 pt
- **Espacio en Blanco**
 - No sobrecarguen el póster.
 - El espacio en blanco ayuda a que la información sea más accesible y menos abrumadora.
- **Gráficos e Imágenes**
 - Asegúrense de que sean de alta calidad y relevantes.
 - Incluyan leyendas para aclarar cualquier gráfico o imagen.
- **Estilo de Redacción**
 - Utilicen oraciones cortas y concisas.
 - Utilicen viñetas para facilitar la lectura y evitar párrafos extensos.
 - El póster debe ser fácil de leer y captar la atención de manera efectiva.

4. Impresión

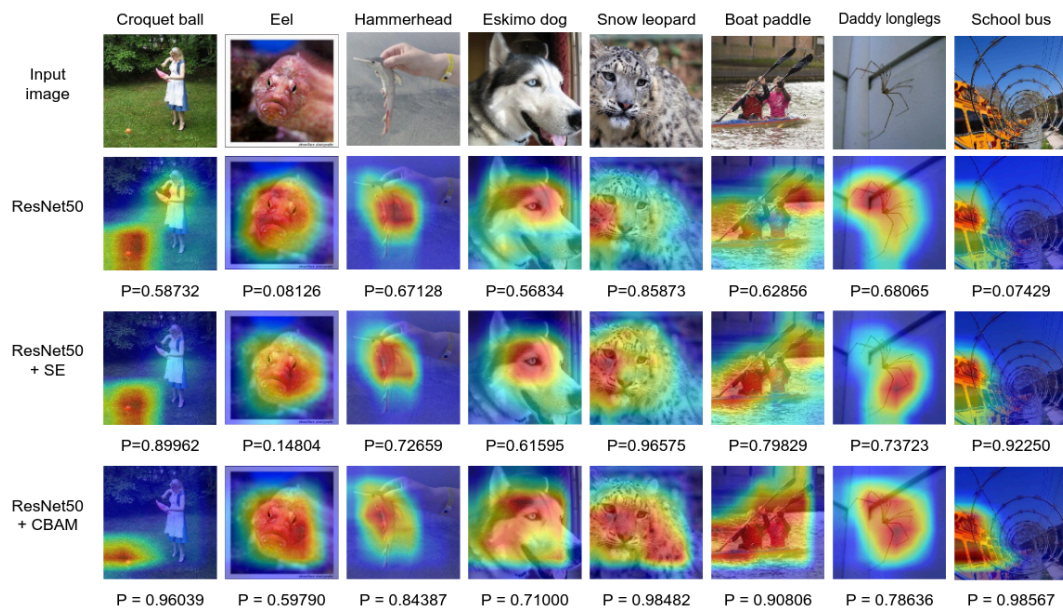
- Impriman en **tamaño A0 (841 x 1189 mm) o A1 (594 x 841 mm) en orientación vertical.**
- Planifiquen la impresión con suficiente antelación para evitar contratiempos.

- La sede del Centro de Copiado “La Copia” del Campus de UdeSA **no realiza impresiones de pósters**. Como alternativa, les sugerimos imprimir el póster en la sede principal del Centro de Copiado “La Copia” en Ciudad Universitaria (ccclacopia@gmail.com), donde estará listo en aproximadamente 2 horas. El horario de atención es de lunes a viernes de 08:00 a 21:00 hs y los sábados de 08:00 a 12:00 hs. Esto solo es una recomendación, pueden optar por imprimir el póster en el lugar que consideren más conveniente.

5. Presentación Oral

- **Preparación:** Practiquen una breve presentación oral (~ 5 min) que resuma los puntos clave de su póster.
- **Interacción:** Estén preparados para responder preguntas y discutir su trabajo con los asistentes de la sesión.

1. Clasificación de Imágenes



Visualizaciones de propagación de gradiente (Grad-CAM),

Woo et al. "[CBAM: Convolutional Block Attention Module](#)". Este paper propone un mecanismo de atención entrenable y compara el desempeño al incluirlo a redes convolucionales conocidas.

La investigación académica tiende a enfocarse en aspectos técnicos novedosos y generalmente muestra resultados en conjuntos de datos de referencia estándar para permitir comparaciones justas con trabajos anteriores.

En el mundo real, los despliegues de visión por computadora a menudo modifican ambas suposiciones: tienden a utilizar ideas simples y comprobadas en lugar de modelos complejos de última generación y suelen utilizar conjuntos de datos personalizados adaptados al problema en cuestión. En muchas aplicaciones prácticas de visión por computadora, la tarea de construir y curar conjuntos de datos para entrenamiento y evaluación toma mucho más tiempo y esfuerzo que el proceso de construcción de los modelos.

De todas las ideas que hemos discutido en clase, la más útil en la práctica es la **clasificación de imágenes mediante aprendizaje por transferencia desde un modelo pre-entrenado**. Si en el futuro aplicas visión por computadora en tu trabajo, esta es probablemente la técnica que utilizarás. En este proyecto construirás un sistema de clasificación de imágenes tal como lo harías en una situación real.

Deberás recolectar tu propio conjunto de datos de clasificación de imágenes (puede ser un *dataset* preexistente de la web), dividirlo en particiones (entrenamiento / validación / prueba) y entrenar modelos de clasificación. Se les pide proponer una arquitectura convolucional diseñada por ustedes y comparar los resultados con un modelo obtenido mediante aprendizaje por transferencia desde redes pre-entrenadas en ImageNet.

Deberán explorar varias opciones y parámetros, y analizar el rendimiento de sus modelos entrenados. Además, existen técnicas avanzadas de visualización, interpretación y exploración de redes convolucionales, estas técnicas se las conoce como de “**Model Understanding**”, siendo el método **Grad-CAM** (Gradient-weighted Class Activation Mapping) uno de los más conocidos. Para este trabajo se les pide, entonces, desarrollar uno o más de estos métodos de interpretación y mostrar resultados sobre alguno de los modelos de clasificación obtenidos.

Links de información general sobre **Grad-CAM** y otros métodos de interpretación:

- La librería [pytorch-grad-cam](#) implementa estos métodos de explicabilidad de modelos y podrán encontrar una introducción al método **Grad-CAM**: [Introduction: Advanced Explainable AI for computer vision](#)
- La librería **Captum** para PyTorch está también implementa varios métodos: <https://pytorch.org/tutorials/beginner/introyt/captumyt.html>

Artículos recomendados

Además de lo visto en clase, estos son dos de los primeros artículos sobre aprendizaje por transferencia para redes convolucionales profundas:

- Donahue et al, “DeCAF: A Deep Convolutional Activation Feature for Generic Visual Recognition”, ICML 2014. [\[arXiv\]](#)
- Razavian et al, “CNN Features off-the-shelf: an Astounding Baseline for Recognition”, CVPR Workshops 2014. [\[arXiv\]](#)

Al analizar un modelo de clasificación entrenado, un método de atribución intenta determinar qué porción de una imagen de entrada fue responsable de la decisión del modelo. Algunos artículos en esta área que podés leer son:

- Simonyan et al, “Deep Inside Convolutional Networks: Visualising Image Classification Models and Saliency Maps”, ICLR Workshop 2014. [\[arXiv\]](#)
- Selvaraju et al, “Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization”, IJCV 2019. [\[arXiv\]](#)
- Fong et al, “Understanding Deep Networks via Extremal Perturbations and Smooth Masks”, ICCV 2019. [\[arXiv\]](#)

Crear un Conjunto de Datos de Clasificación de Imágenes

La primera tarea en este proyecto es recolectar tu propio conjunto de datos de clasificación de imágenes para una tarea de clasificación que elijas.

Primero deberían definir un conjunto de dos o más categorías en las que quieran clasificar las imágenes. Estas categorías pueden ser amplias (p. ej., perro/gato/pez, hotdog/no-hotdog) o más específicas (por ejemplo, perro siberiano/perro salchicha).

¡Pónganse creativos con las categorías que elijan!

Decidan si sus categorías son mutuamente excluyentes (una imagen tendrá exactamente una etiqueta correcta) o no excluyentes (una imagen puede tener cero o más etiquetas correctas). Las categorías mutuamente excluyentes son más simples de manejar y analizar, por lo que te sugerimos que intentes idear un conjunto de categorías mutuamente excluyentes.

Una vez que hayan decidido un conjunto de categorías, necesitarán construir un conjunto de datos recolectando y etiquetando imágenes. Los conjuntos de datos más grandes suelen dar mejores resultados., El conjunto de datos debería tener al menos 100 imágenes por categoría.

[Google Dataset Search](#) es una herramienta moderna muy interesante que indexa conjuntos de datos de varios sitios (kaggle, roboflow, mendeley, figshare, etc). Ejemplo de búsqueda, [fruit classification](#).

Pueden explorar la librería [Selenium](#) para “scrapear” imágenes de la web, algunos links útiles: [How to Scrape Images from Google Using Python Selenium](#), [End-to-End ML](#), [A simple Selenium image scrape from an interactive Google Image Search](#)

Usen la creatividad en la fuente de sus imágenes, pueden buscar imágenes en internet, mezclar datasets o armar un dataset propio tomando fotos.

Luego de recolectar y etiquetar tu conjunto de datos, deberían dividirlo en conjuntos de entrenamiento, validación y prueba. Una regla general para dividir conjuntos de datos es asignar el 70% de tus datos a entrenamiento, 10% a validación y 20% a prueba.

Tengan en cuenta que el proceso de construir y etiquetar el conjunto de datos constituye una gran parte del trabajo para este proyecto.

En todos los casos el modelo final debe ser probado con imágenes nuevas no contenidas en el conjunto de datos.

El informe debería incluir una sección que describa la tarea de clasificación y el conjunto de datos. Expliquen qué tarea de clasificación están tratando de resolver, por qué podría ser útil y cuáles son las categorías. Describan el proceso de recolección y etiquetado de imágenes, y discutan los desafíos que enfrentaron al construir el conjunto de datos. Finalmente, muestren ejemplos del conjunto de datos para dar una idea de los tipos de imágenes que contiene.

Entrenar un Clasificador de Imágenes

Una vez que hayan recolectado un conjunto de datos, es hora de entrenar varios modelos con ese conjunto. Pueden proponer una arquitectura de red neuronal diseñada por ustedes mismos, pero tengan cuidado con el tiempo que puede llevar esto.

Pueden utilizar torchvision, que proporciona [una amplia variedad de modelos pre entrenados](#). Les recomendamos comparar múltiples arquitecturas. Por ejemplo ResNet18, ResNet50 o un modelo pequeño de RegNet (p. ej., RegNetY-400MF). Pueden considerar tres maneras diferentes de entrenar un clasificador:

- Entrenar desde cero: Inicializar el modelo aleatoriamente y entrenarlo desde cero en un dataset de entrenamiento.
- Transfer learning: Utilizar un modelo con pesos pre-entrenados (ejemplo sobre ImageNet); que permitirá extraer características visuales. Luego utilizar clasificador sobre dichas características visuales, para resolver otro problema distinto al que fue entrenando.
- Fine-Tuning: Inicializar el modelo usando pesos pre entrenados en ImageNet; y ajustar una porción (o todo) el modelo en el conjunto de entrenamiento. Esto debería funcionar al menos tan bien como con Transfer Learning.

Es posible que necesiten experimentar con diferentes optimizadores, tasas de aprendizaje, regularización L2 y estrategias de **data augmentation**.

Deberán entrenar al menos 6 variantes de modelos diferentes y reportar su precisión tanto en los conjuntos de entrenamiento como de validación.

Deberán medir y presentar el **tiempo de latencia en inferencia** para cada una de las variantes presentadas.

Deberán **graficar las curvas de las métricas de entrenamiento** de los modelos (**loss y accuracy**) vs **epoch**, para los splits de train y validation. Deberán reportar las métricas predictivas en el split de test.

El informe deberá explicar los modelos que eligieron, describir los experimentos que realizaron y mostrar los resultados descritos arriba.

Analizar los Modelos

Después de haber entrenado los modelos, deberán analizarlos para intentar obtener información sobre qué tan bien están funcionando y en qué situaciones fallan. Deberán analizar el modelo con mejor rendimiento y opcionalmente algún otro modelo de interés.

Deberán mostrar lo siguiente:

- Ejemplos cualitativos de imágenes que fueron clasificadas correctamente e incorrectamente. Pueden mostrar tanto imágenes seleccionadas manualmente para resaltar características interesantes del modelo, como imágenes seleccionadas al azar para dar una mejor idea del rendimiento promedio del modelo.
- Matriz de confusión: Una matriz 2D de categoría predicha versus categoría real, donde cada entrada muestra la fracción de imágenes del conjunto de validación que caen en esa situación; esto muestra los tipos de errores que comete el modelo.
- Métodos de atribución: Usar uno o más métodos de atribución para dar ejemplos de qué porciones de las imágenes utiliza el modelo para tomar decisiones de clasificación.
- Graficar las características visuales extraídas por la red en las capas previas a la clasificación. Pueden utilizar métodos como reducción de la dimensionalidad PCA, o la técnica de t-SNE (t-distributed stochastic neighbor embedding, [Mastering t-SNE](#)).

El informe debería incluir secciones que describan y muestren cada uno de los métodos de análisis anteriores, así como cualquier otro tipo de análisis que realicen.

Evaluación en el Conjunto de Prueba

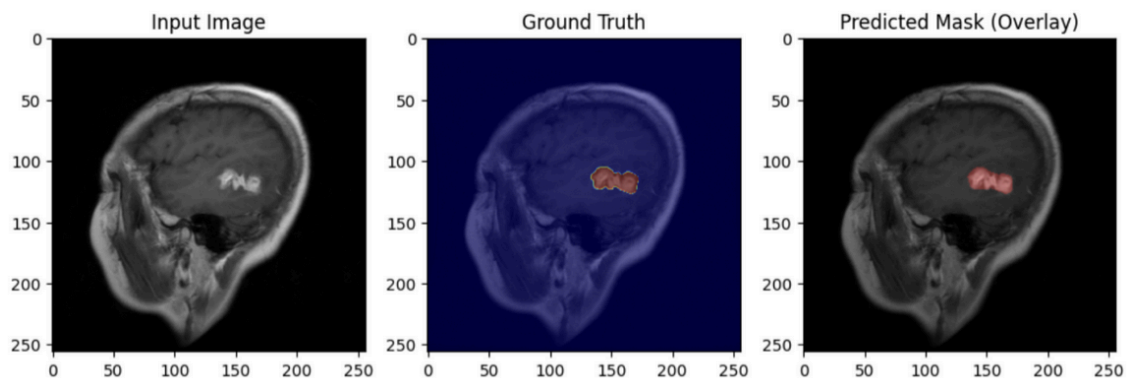
Después de todo el análisis, deberían correr el mejor modelo en el conjunto de prueba. Si el modelo tiene un rendimiento muy diferente en los conjuntos de validación y prueba, deberían intentar explicar por qué podría ser ese el caso.

Entregables

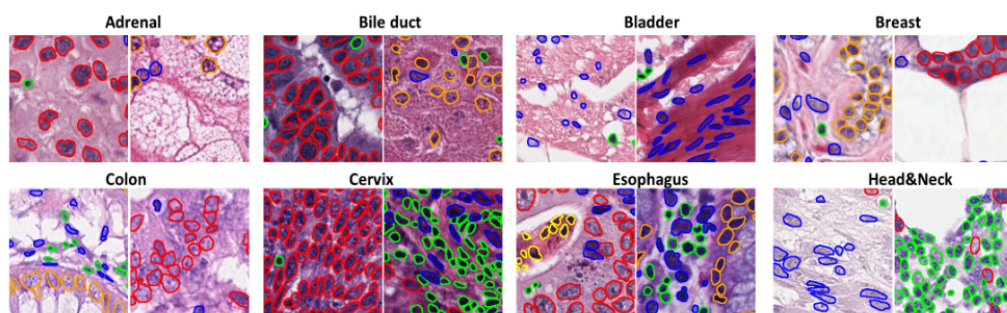
En resumen, en este proyecto esperamos que:

- Creen su propio conjunto de datos de clasificación de imágenes con al menos 100 imágenes por categoría. En el informe, discutan el problema de clasificación que eligieron resolver y cómo recolectaron el conjunto de datos. Muestren imágenes de ejemplo del conjunto.
- Entrenen al menos 6 variantes de modelos de clasificación de imágenes sobre el conjunto de datos; muestren curvas de entrenamiento para cada modelo (pérdida y precisión), así como una comparación de precisión en el conjunto de validación. Tiempos de inferencia.
- Analicen el rendimiento de los modelos mostrando ejemplos de clasificación correcta/incorrecta, matrices de confusión y métodos de atribución.
- Prueben el mejor modelo hallado sobre el conjunto de prueba y discutan los resultados.

2. Segmentación de patologías en imágenes médicas



Segmentación de células tumorales en el cerebro. Imágenes del Kaggle Brain Tumor Dataset.



Segmentación de células neoplásicas (tumorales) y no neoplásicas (no tumorales) en diferentes tejidos del cuerpo humano. Imágenes del PanNuke Dataset.

Uno de los casos de aplicación más exitosos de las redes convolucionales es la de segmentación de imágenes médicas para detección de patologías. Numerosos estudios han demostrado el potencial de los modelos de aprendizaje profundo en la detección de cáncer, clasificación de tejidos, identificación de diagnósticos estructuras llamativamente relevantes e incluso infiriendo subtipos genéticos.

La particularidad de este tipo de problemas es que, a diferencia de la clasificación, se busca asignar una etiqueta a cada píxel de la entrada.

Con los años se han propuesto diferentes arquitecturas y estrategias para resolverlo, en este trabajo deberán explorar dos arquitecturas fundacionales:

1. Uno de los primeros enfoques para realizar la segmentación semántica mediante una red neuronal profunda se propuso en el artículo de CVPR de 2015 [Fully Convolutional Networks for Semantic Segmentation](#) (FCN, Redes totalmente convolucionales para la segmentación semántica).

2. Del desarrollo y mejora de las redes FCN, se propuso luego una de las arquitecturas convolucionales más exitosas, la arquitectura conocida como [U-Net: Convolutional Networks for Biomedical Image Segmentation](#)

La arquitectura U-Net fue de gran impacto en la comunidad y trajo consigo una gran explosión de propuestas han surgido en la historia de las redes neuronales. Es posible mencionar la [DeepLabV3](#) que propone una alternativa a la transposed convolution y las conexiones residuales.

Y muchas otras cómo la [TernausNet: U-Net with VGG11 Encoder Pre-Trained on ImageNet for Image Segmentation](#) que se enfoca en aplicar Transfer Learning utilizando una red VGG.

Para este trabajo deberán seleccionar un *dataset* de segmentación y analizar el desempeño de al menos dos arquitecturas cómo las mencionadas anteriormente y proponer una variación diseñada por ustedes.

Conjuntos de datos

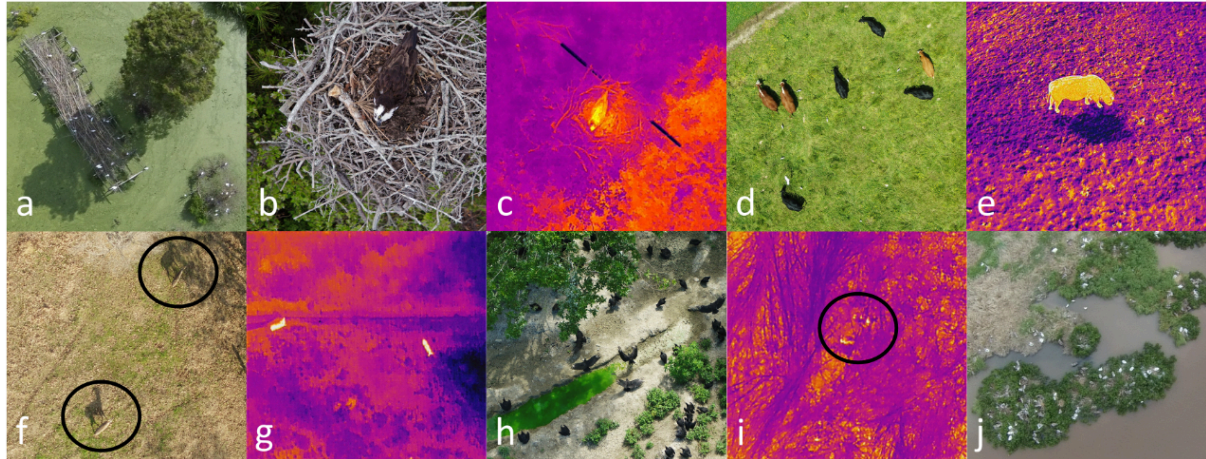
La cátedra recomienda dos *datasets* de segmentación de imágenes:

1. Figshare Brain Tumor Dataset (una alternativa de **tamaño moderado**):
El conjunto de datos contiene 3064 pares de imágenes de resonancia magnética cerebral y su respectiva máscaras binarias que indican el tumor:
[Repositorio Figshare](#)
[Repositorio Kaggle](#)
2. PanNuke Dataset: Uno de los *datasets* más grandes y mejor organizados con 205.343 células cancerígenas marcadas en varios diferentes tipos de tejidos:
[Repositorio Kaggle](#)
[Torrent libre del dataset](#)
[Paper de su última extensión en 2020](#)
[Presentación del trabajo en conferencia internacional](#)

Alternativamente pueden buscar algún otro que les sea de interés. Pueden inspeccionar soluciones preexistentes que les faciliten el tratamiento y procesamiento de los datos, en todos los casos deben citar fuentes y analizar las metodologías aplicadas.

En caso de obtener buenos resultados pueden proponer su solución en Kaggle.

3. Monitoreo de vida silvestre utilizando Imágenes RGB y termales



Imágenes del Aerial Wildlife Image Repository

La utilización de drones (aeronaves no tripuladas) se han convertido en herramientas eficaces para el monitoreo y la conservación de la vida silvestre. La detección y clasificación automatizada de animales mediante inteligencia artificial (IA) ha demostrado ser muy precisa y reduce sustancialmente los costos logísticos.

En este trabajo deberán trabajar sobre el conjunto de datos [Aerial Wildlife Image Repository](#) de la universidad de Mississippi. Este conjunto de datos está compuesto por tres subconjuntos para el monitoreo de tres tipos de animales distintos: vacas, ciervos y caballos; y está compuesto por una colección de imágenes RGB, imágenes termales y coordenadas de los animales en las imágenes etiquetadas por biólogos expertos.

Las cámaras utilizadas para conformación del dataset son cámaras estéreo con dos sensores ([DJI Zenmuse XT2](#)), un sensor de imágenes RGB y otro Termal, por este motivo se lo conoce como un conjunto de datos RGB-T. Las cámaras fueron calibradas previamente y el conjunto de datos provee imágenes rectificadas, es decir, las imágenes termales se encuentran *warpeadas* para coincidir perfectamente con las imágenes RGB (un determinado objeto/animal visto en la imagen RGB se encontrará en la misma coordenada en la correspondiente imagen Termal).

Detección de objetos utilizando YOLO

Para este trabajo deberán implementar una solución utilizando la famosa arquitectura de detección de objetos **YOLO** introducida por Redmon, Joseph y Farhadi, Ali en 2018, [You Only Look Once: Unified, Real-Time Object Detection](#).

A modo histórico les dejamos el enlace a la web original mantenida por Redmon, <https://pjreddie.com/darknet/yolo/>, el cual siempre tuvo un atractivo particular para la comunidad hacktivista, destacando el tono humorístico de Redmon en sus publicaciones.

Con los años la arquitectura YOLO fue cambiando y se fue mejorando por la comunidad, siendo hoy [Ultralytics Roboflow](#) el principal mantenedor:

[What is YOLO? The Ultimate Guide.](#)

La infraestructura open-source mantenida por Ultralytics provee **facilidades para la aumentación de datos y versiones del modelo optimizadas** para móviles y cómputo embebido.

Se advierte que **versiones posteriores a la YOLOv8 podrían presentar componentes en su arquitectura que no hemos visto en la materia.**

Material de referencia de la YOLOv11: [How to Train YOLOv11 on a Custom Dataset](#)

La particularidad del trabajo reside en modificar YOLO para trabajar con imágenes RGB y Termal, de manera de mejorar la detección de objetos correlacionando esta información.

YOLO RGB+T

El trabajo científico: [Adopting the YOLOv4 Architecture for Low-Latency Multispectral Pedestrian Detection in Autonomous Driving](#) analiza la adaptación de la red YOLOv4 (solo RGB) para la detección de peatones utilizando imágenes RGB y Termal.

El trabajo realiza un estudio exhaustivo de varias alternativas para la adaptación de la arquitectura YOLOv4.

Primeramente analiza la implementación de una versión **YOLOv4-T** (Sección 3.2, *YOLOv4 with Thermal Images*) que considera únicamente las imágenes termal en una versión de YOLO de 1 canal. Esto es posible modificando la cantidad de canales de los filtros de la primera capa convolucional (de 3 canales a solo 1). Pero esto no considera las imágenes RGB.

Luego el artículo propone una serie de alternativas para fusionar la información RGB y Termal:

1. **Enfoque Early Fusion (YOLO-HST/GST):** Comprime la información RGB-T en solo tres canales de manera de conformar con el pipeline habitual de entrenamiento de la red de 3 canales.

2. **Enfoque *Late Fusion*:** Aplica las versiones YOLO4-RGB y YOLO4-T (de 3 canales ambas) en paralelo para comparar el solapamiento de las detecciones al final del *pipeline*.
3. **YOLO-RGB-T:** La información termal es incluida en un cuarto canal de información, se realiza el entrenamiento reutilizando los pesos de la red YOLO-RGB original para todas las partes comunes de la arquitectura.
4. **Enfoque *Middle Fusion*:** Finalmente propone una solución superadora con un mayor grado de complejidad correlacionando la información Termal en capas intermedias de la red.

Los autores proveen solo la implementación de la solución más compleja llamada [YOLOv4-Middle](#).

NO ES NECESARIO REALIZAR UNA IMPLEMENTACIÓN DE ESTA SOLUCIÓN PARA LA APROBACIÓN DEL TRABAJO. Pueden considerar opcionalmente, plantear una idea similar más sencilla.

Se aconseja idear una metodología de exploración del problema aplicando en orden progresivo de complejidad las soluciones:

YOLO-HST/GST < YOLO-T < YOLO Late-F < YOLO-RGB-T < YOLO Middle-F

Se recomienda implementar los diferentes enfoques en versiones modernas del YOLO del [repositorio oficial mantenido por ultralytics](#) que, recientemente, [soporta información multiespectral](#).

Otros conjuntos de datos con imágenes termales

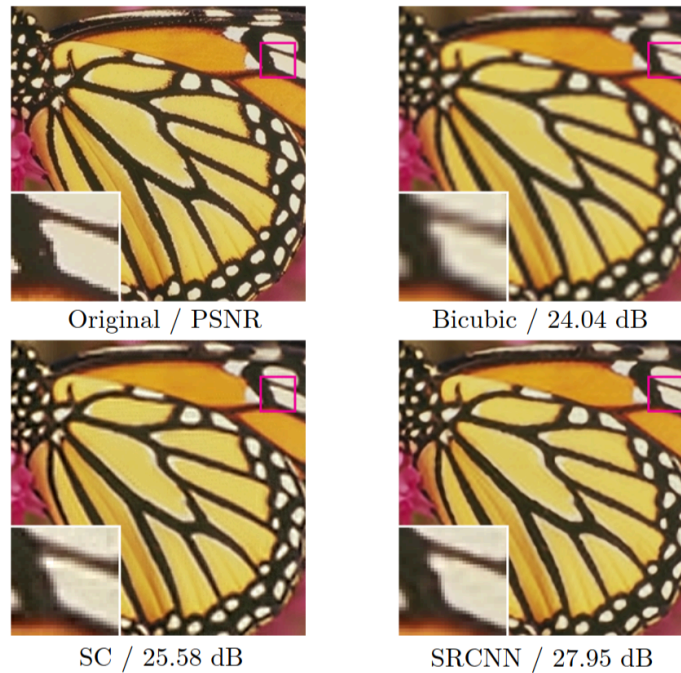
Opcionalmente pueden buscar otro tipo de conjunto de datos con imágenes multiespectrales de algún tipo.

Por ejemplo, el [VisDrone DroneRGBT Dataset](#).

Comparación de desempeño con otros modelos

Deberán comparar el desempeño entre las diferentes soluciones basadas en YOLO. Opcionalmente, pueden comparar el desempeño con modelos “más clásicos” como el Faster-RCNN y los propuestos en la [librería torchvision](#). Pueden además considerar librerías como [RF-Detr](#) que provee una implementación eficiente de detector de objetos basado en Transformers y [Detectron2 de Meta](#) la cual tiene implementaciones de varias arquitecturas de redes de detección de objetos modernas.

4. Super-Resolución de Imágenes



El método Super-Resolution Convolutional Neural Network (SRCNN) propuesto en Dong et al. "Image super-resolution using deep convolutional networks". Superando ampliamente al desempeño de métodos como el interpolador Bicúbico.



Métodos de DLSS (Deep Learning Super Sampling) de Nvidia que permiten ganar performance procesando gráficos a baja resolución para luego escalarlos utilizando modelos, se puede observar que mantiene calidad visual a mucho mayor FPS.

En la tarea de super-resolución de imágenes individuales, una red recibe como entrada una imagen de baja resolución y genera como salida una versión de mayor resolución de la misma (**¡una imagen más grande!**).

Este es en general, un problema sub-especificado, es decir, hay muchas posibles imágenes de alta resolución que podrían corresponder a la misma imagen de baja resolución. Sin embargo, al entrenar en conjuntos de datos de imágenes de alta y baja resolución correspondientes, una red neuronal puede aprender patrones sobre el mundo visual que le permiten predecir una salida plausible de alta resolución para cualquier entrada de baja resolución.

De forma similar a la segmentación semántica, se puede usar una red totalmente convolucional (*fully-convolutional network*) para construir un modelo de super-resolución de imágenes individuales.

La entrada del modelo es una imagen de baja resolución de forma $3 \times H \times W$; la imagen pasa por una serie de capas de convolución y capas de aumento de escala (usando interpolación de vecino más cercano, bilineal, o convolución traspuesta) y finalmente produce una salida de alta resolución de la forma $3 \times fH \times fW$, donde f es el factor de super-resolución (en general tenemos que $f=2$, $f=3$ o $f=4$). Durante el entrenamiento, la salida de la red se compara con la salida real usando una función de pérdida L2, L1 o alguna otra.

El objetivo de este proyecto es implementar y entrenar una red convolucional para super-resolución de imágenes individuales. Se espera que entrenen modelos para al menos dos factores de super-resolución ($f=2$ y otro a elección).

Para esto deberán diseñar y proponer una arquitectura inspirándose en artículos científicos académicos. Deberán mostrar resultados de entrenamiento y validación utilizando métricas particularmente relevantes para este tipo de tarea específica. Les recomendamos seguir el siguiente trabajo científico como guía metodológica:

SRCNN: Dong et al, "Image super-resolution using deep convolutional networks", TPAMI 2015. [[arXiv](#)]

Artículos recomendados

A continuación listamos artículos fundacionales de la técnica de super-resolución, cada uno propone una nueva metodología y están en orden ascendente de complejidad:

- **SRCNN**: Dong et al, "Image super-resolution using deep convolutional networks", TPAMI 2015. [[arXiv](#)]
- **FSRCNN**: Dong et al, "Accelerating the super-resolution convolutional neural network", ECCV 2016. [[arXiv](#)]

- **SRGAN**: Ledig et al, “Photo-Realistic Single Image Super-Resolution using a Generative Adversarial Network”, CVPR 2017. [[arXiv](#)]

Pueden buscar otras soluciones en las cuales inspirarse. Se recomienda comenzar copiando una de las arquitecturas más sencillas y luego probar otras arquitecturas.

Conjunto de datos

Pueden entrenar un modelo en cualquier conjunto de datos de imágenes que elijan; los conjuntos de datos de ImageNet o COCO son opciones bastante populares. La super-resolución generalmente no requiere conjuntos de datos muy grandes para el entrenamiento.

Por ejemplo, **FSRCNN** se entrena en un conjunto de datos de 100 imágenes llamado General100 (<http://mmlab.ie.cuhk.edu.hk/projects/FSRCNN.html>), que podría resultarles útil para una implementación propia. Consulten la Sección 4.1 del artículo de **FSRCNN** para obtener más detalles sobre cómo preparar el conjunto de datos de entrenamiento.

Deberán evaluar su modelo en los conjuntos de datos **Set5** y **Set14**. Durante la evaluación, la manera precisa en la que preparan las imágenes de baja y alta resolución es importante; por esta razón, deberían usar las versiones de los conjuntos de datos **Set5** y **Set14** de [este repositorio de GitHub de Jia-Bin Huang](#). Tengan en cuenta que existen versiones diferentes de los conjuntos de datos para cada factor de super-resolución.

Modelo

Deberían implementar al menos un modelo similar a **SRCNN** o **FSRCNN**; sigan los detalles en los artículos lo más cerca posible.

Se recomienda entrenar el modelo primero con el objetivo de obtener imágenes al doble de escala $f=2$, lograr convergencia del modelo en ese escenario y luego intentar factores de escala mayores como $f=4$, etc..

Como una extensión opcional, también podrían considerar usar una red generativa adversarial para super-resolución como se describe en el artículo de SRGAN. Entrenar el modelo adversarialmente contra un discriminador aprendido puede producir imágenes de mayor calidad perceptual, aunque con un PSNR o SSIM ligeramente inferior en comparación con un modelo entrenado solo con pérdidas L2 o L1.

El informe debe incluir una sección que describa la arquitectura del modelo, las funciones de pérdida de entrenamiento y cualquier otro detalle de implementación importante de tu modelo. Deberían experimentar con distintos

optimizadores, funciones de pérdidas, arquitecturas de modelo u otras variaciones que se les ocurran. Deberían mostrar los resultados de al menos **dos** variantes diferentes del modelo para cada factor de super-resolución que consideren.

Evaluación

Deberían evaluar el modelo tanto en imágenes no vistas de tu conjunto de entrenamiento como también en los conjuntos de datos Set5 y Set14. Pueden opcionalmente elegir reportar resultados adicionales en los conjuntos de datos BSD100 o Urban100.

Para este trabajo deberán trabajar con dos métricas específicas de relación señal-ruido pico (PSNR) y la similitud estructural (SSIM). **Lean con atención los artículos científicos con respecto a esto**, pueden encontrar [implementaciones de estas métricas en scikit-image](#).

Deberían comparar tu modelo con el aumento de escala usando interpolación bilineal, bicúbico y de vecinos más cercanos; pueden usar OpenCV o scikit-image para esto. El mejor modelo debería superar ambos métodos de referencia en los factores de super-resolución que consideren.

Concretamente, para un factor de super-resolución de 2 deberían alcanzar un PSNR de al menos 34 en Set5; para un factor de super-resolución de 3 deberían alcanzar un PSNR de al menos 31 en Set5; y para un factor de super-resolución de 4 deberían alcanzar un PSNR de al menos 29 en Set5.

Canales de Color

Un detalle importante de los métodos de super-resolución es el canal de color que el modelo utiliza para el entrenamiento y la evaluación. Hasta ahora en este curso, todas las imágenes se han representado en el espacio de color RGB estándar; sin embargo, existen otros espacios de color que se utilizan comúnmente para representar imágenes. Uno importante es el espacio de color YCbCr. En este espacio de color, Y es un canal de color en escala de grises (luma) y Cb y Cr son dos canales que dan el color de la imagen (croma). Podés convertir entre RGB y YCbCr usando funciones de OpenCV o scikit-image.

Para la super-resolución de imágenes individuales, una configuración común es transformar la imagen del espacio de color RGB al espacio de color YCbCr; la red neuronal recibe el canal Y de baja resolución y predice un canal Y de alta resolución, mientras que los canales Cb y Cr se amplían mediante interpolación bicúbica. La pérdida de entrenamiento compara los canales Y predicho y el real, y durante la evaluación calculamos PSNR y SSIM entre los canales Y predicho y real.

Deberían seguir esta convención en la implementación. También es posible entrenar modelos de super-resolución usando entradas y salidas en color; para

más detalles, consulten la Sección 4.5 del artículo de SRCNN. Pueden realizar experimentos adicionales con canales de color adicionales de forma opcional.

Conclusión

Para resumir, en este proyecto se espera que:

- Implementen modelos para super-resolución de imágenes individuales. Deberías entrenar modelos para al menos dos factores de super-resolución diferentes (2x, 3x o 4x). Para cada modelo deberías mostrar curvas de entrenamiento (pérdida en el conjunto de entrenamiento y PSNR en un conjunto de validación no visto a lo largo del entrenamiento). Para cada modelo deberías reportar el rendimiento en los conjuntos de datos Set5 y Set14 usando PSNR y SSIM.
- Deberían comparar tu modelo con el aumento de escala con interpolación bilineal, bicúbico y vecinos más cercanos, y tu modelo debería superar estos métodos de referencia.
- Para factores de super-resolución 2x / 3x / 4x, deberías lograr PSNR en Set5 de al menos 34 / 31 / 29.
- También deberían mostrar resultados cualitativos. Por ejemplo, podés mostrar la imagen de baja resolución, la imagen de alta resolución real y la imagen de alta resolución predicha para cada método que consideres (y los métodos bilineal / bicúbico / vecinos más cercanos) para varios ejemplos de un conjunto de validación no visto, así como para cada imagen en Set5.

5. Colorización de imágenes



Colorización de imágenes antiguas en blanco y negro por el método propuesto en “Let there be Color!: Joint End-to-end Learning of Global and Local Image Priors for Automatic Image Colorization with Simultaneous Classification” de Izuka et al.

La **colorización automática** es el proceso de asignar colores plausibles a una imagen en escala de grises sin intervención humana. Este problema es intrínsecamente ambiguo (por ejemplo, un vestido podría ser de muchos colores), pero las pistas contextuales ayudan: el cielo suele ser azul, la hierba verde, etc. En el año 2016 los avances en las arquitecturas de redes neuronales generó una sucesión de trabajos que plantearon soluciones muy eficientes al problema de colorización automática.

Para solucionar el problema es usual **utilizar el espacio de color CIE $L^*a^*b^*$** , un espacio de tres canales llamados: L^* , a^* , b^* , donde L^* es una correlación de la luminosidad (similar al concepto de intensidad) y a^* , b^* son denominados ejes de colores complementarios, que representan (aproximadamente) los componentes de rojo (positivo) y verde (negativo). Este espacio de color fue diseñado de manera tal que si dos puntos en éste espacio son coincidentes, entonces la diferencia cromática entre ambos estímulos es igual a cero y, según se incrementa la distancia, entre esos dos puntos aumentará la percepción de diferencia cromática entre los estímulos que ambos puntos representan (desde la perspectiva de la percepción de un humano promedio).

Cada imagen de color en el conjunto de entrenamiento es convertida a escala de grises y al espacio CIE $L^*a^*b^*$. La entrada del modelo será entonces la imagen escala de grises y deberá inferir los valores de los canales a^* y b^* para poder luego reconstruir la imagen a color (con el canal de grises como L , a^* y b^*).

La **función de costo MSE (Mean Squared Error)** es una función de costo sencilla que puede utilizarse para establecer la diferencia entre los valores reales e inferidos.

Un enfoque interesante en dichos años es el presentado por Larsson et al., donde utiliza **redes preentrenadas para extracción de características “en cascada”** de diferentes capas de la red (capas “congeladas”), para luego entrenar únicamente una capa densamente conexas que transforme las características a los valores requeridos. Utiliza la función de costo MSE pero además propone unas mejoras en base a análisis de histogramas.

Otros enfoques notan que función de costo MSE da resultados subóptimos (“aguados”). Y proponen, entonces, agregar **una segunda función de costo de clasificación** que provea información semántica a la red sobre el contexto global de la imagen.

A continuación les referenciamos los trabajos fundacionales mencionados.

Artículos recomendados

- Zhang et al., “Colorful image colorization”. ECCV 2016. [[arXiv](#)]
- Iizuka et al., “Let there be Color!: Joint End-to-end Learning of Global and Local Image Priors for Automatic Image Colorization with Simultaneous Classification”. ACM Trans. Graph 2016. [[web](#)]
- Larsson, et al., “Learning Representations for Automatic Colorization”. ECCV 2016. [[arXiv](#)]

Modelos

Deberán referenciarse a los trabajos citados para proponer al menos dos arquitecturas distintas comparando desempeño al utilizar al menos dos políticas de función de costo.

Conjunto de datos y evaluación

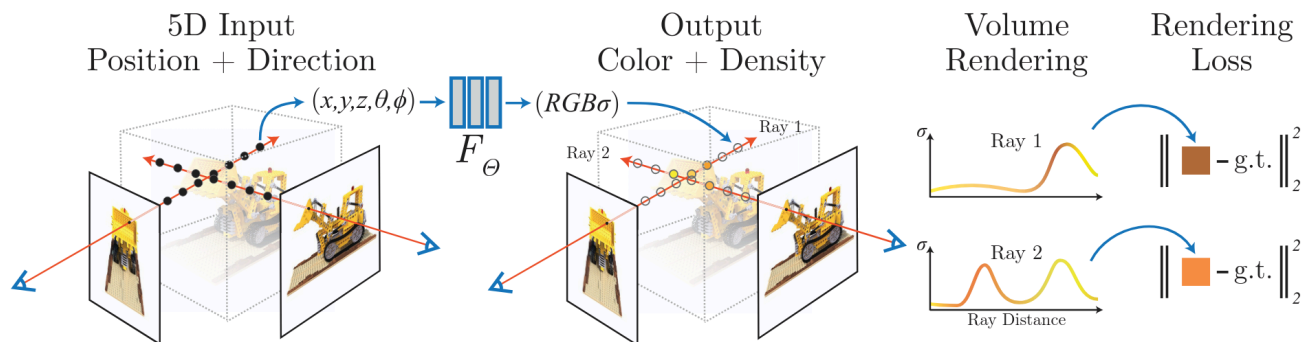
Para entrenar y evaluar los modelos solo es necesario utilizar colecciones de imágenes a color. Se sugiere primero comenzar con *subconjuntos manejables* de grandes repositorios: por ejemplo, el recorte [Fast.ai imagenette/imagewoof](#) del ImageNet o el dataset [Places](#), antes de intentar trabajar con el ImageNet entero (o una mezcla de varios). En todos los casos deben indicar cómo se prepararon las imágenes (reescalado, normalización, conversión al espacio CIELAB, etc.) y dividir los datos en particiones de entrenamiento, validación y prueba.

Los resultados obtenidos deben evaluarse tanto cualitativa (visualización de colorizaciones resultantes) como cuantitativamente (por ejemplo, precisión media de color o métricas de percepción visual como la relación señal-ruido pico (PSNR) y la similitud estructural (SSIM).

Deberán referenciarse a los trabajos citados para replicar la metodología de evaluación y el estilo de tablas de comparación. No olviden reportar los comportamientos de las funciones de costo utilizadas y su convergencia durante las iteraciones de entrenamiento.

¡Si tienen alguna foto familiar antigua pueden colorearla!

6. Síntesis de vistas utilizando NeRFs



Revolucionario método propuesto de Campos de Radiancia Neuronal (*Neural Radiance Fields, NeRF*) utilizado para realizar reconstrucciones 3D y síntesis de “nuevas vistas” utilizando imágenes y poses estimadas de cámaras. Trabajo presentado en Mildenhall et al. “Representing Scenes as Neural Radiance Fields for View Synthesis”

En la tarea de **síntesis de nuevas vistas (views)**, el conjunto de entrenamiento consiste en imágenes de una escena cuyos parámetros de cámara (intrínsecos y extrínsecos) son conocidos. El objetivo es crear un modelo capaz de sintetizar imágenes que muestren la escena desde perspectivas nuevas, no vistas en el conjunto de entrenamiento.

En los últimos dos años, los Campos de Radiancia Neuronal (*Neural Radiance Fields, NeRF*) se han consolidado como un modelo sencillo y potente para este problema. Los NeRF se basan en el concepto de renderizado volumétrico: para determinar el color de un píxel, se proyecta un rayo desde el centro de la cámara a través del píxel y hacia la escena; para un conjunto de puntos dado a lo largo de este rayo, se calculan tanto el color como la opacidad (transparencia) de la escena 3D. La integración de la influencia de estos puntos proporciona el color del píxel final en la nueva imagen.

El artículo original de NeRF [Mildenhall et al, ECCV 2020] propone entrenar una red neuronal densamente conexa que recibe como entrada posiciones (x, y, z) y ángulos de rotación de cada vista (es decir, cada imagen), y genera como salida el color RGB y la opacidad de la escena 3D en ese punto. Esta red se entrena para reproducir los valores de píxeles de las imágenes de entrenamiento; durante la inferencia, se puede utilizar para sintetizar el color de los píxeles en nuevas vistas no vistas durante el entrenamiento.

El objetivo de este proyecto es implementar un modelo NeRF y reproducir algunos de los resultados principales del artículo original. Opcionalmente, también se pueden incorporar ideas de artículos posteriores.

Cabe destacar que este proyecto probablemente implique una implementación más compleja que otros proyectos sugeridos. Además, se pretende que sea un proyecto abierto; si bien el objetivo es reimplementar NeRF, se puede ser creativo en cuanto a los resultados que se muestren y en cómo se diferencien del artículo original.

Artículos recomendados

- **NeRF:** Mildenhall et al., “NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis”, ECCV 2020. [[Web](#)] [[youtube](#)] [[arXiv](#)]
- **Mip-NeRF:** Barron et al., “Mip-NeRF: A Multiscale Representation for Anti-Aliasing Neural Radiance Fields”, ICCV 2021. [[arXiv](#)]

Modelo

Deberán reimplementar el modelo del artículo original de NeRF. **Este modelo tarda aproximadamente entre uno y dos días en converger en una sola GPU V100**, por lo que una reimplementación exacta probablemente no sea factible con los recursos computacionales disponibles.

Pueden considerar las siguientes simplificaciones o modificaciones al modelo NeRF original que permitirían un entrenamiento más rápido a costa de obtener resultados de menor calidad:

- **MLP más pequeño:** El artículo utiliza un MLP de 8 capas con 256 unidades ocultas por capa (véase el Apéndice A del paper). Usar un modelo menos profundo o más estrecho podría permitir una convergencia más rápida.
- **Sin muestreo jerárquico:** Como se explica en la sección 5.2 del paper, NeRF entrena modelos gruesos y finos por separado. Primero, muestrea puntos uniformemente a lo largo de los rayos mediante un muestreo estratificado y evalúa el modelo grueso en dichos puntos; las densidades predichas se utilizan luego para muestrear un segundo conjunto de puntos en regiones de alta densidad, que serán evaluados por el modelo fino. Para simplificar y acelerar la implementación, pueden usar solo un modelo grueso.
- **Menos muestras por rayo:** NeRF entrena con 64 muestras “gruesas” y 128 muestras “finas” por rayo; usar menos muestras por rayo durante el entrenamiento puede acelerarlo.
- **Entrenamiento más corto:** NeRF entrena con un tamaño de lote de 4096 rayos y entre 100 000 y 300 000 iteraciones; es posible obtener resultados satisfactorios con un tamaño de lote menor o con menos iteraciones.
- **Imágenes de menor resolución:** NeRF utiliza imágenes de resolución relativamente alta para el entrenamiento: imágenes de 800x800 para escenas sintéticas e imágenes de 1008x756 para escenas reales. En lugar de usar estas imágenes en su resolución original, reducir su resolución durante

el entrenamiento (por un factor de 2x, 4x u 8x) probablemente permitirá una convergencia más rápida.

Si se sienten con ganas de experimentar, también pueden incorporar ideas de Mip-NeRF a su modelo o probar ideas novedosas de tu propio diseño. Describan en el informe las variantes de modelos con las que experimenten. Pueden comparar el tiempo de ejecución y la velocidad de convergencia (grafica el PSNR del conjunto de prueba frente a la época de entrenamiento) para las diferentes variantes de modelo que implementes.

Evaluación

Deben mostrar los resultados en al menos dos escenas diferentes utilizando los conjuntos de datos del artículo de NeRF, disponibles [aquí](#). Pueden también utilizar las imágenes y datos tomados para la reconstrucción 3D del TP 2 de la materia (consideren que la calidad de la estimación de pose afecta mucho la reconstrucción).

Deben evaluar sus resultados utilizando la relación señal-ruido pico (PSNR) y la similitud estructural (SSIM) en imágenes del conjunto de prueba. Probablemente no podrán igualar el rendimiento reportado en el artículo de NeRF, pero aun así debe compararlos como límite superior de su rendimiento.

Pueden también compararse con implementaciones más robustas y modernas cómo el [NerfStudio](#) y [GSplatStudio](#).

Conclusión

En resumen, en este método esperamos que usted:

- **Reimplementen el método** del artículo de NeRF. En lugar de reimplementarlo exactamente, deberá utilizar algunas de las modificaciones sugeridas (u otras de su propio diseño) para intentar reducir el tiempo de entrenamiento. Deberá describir detalladamente cualquier cambio que realice en el algoritmo básico de NeRF.
- **Deberán mostrar resultados** en al menos dos escenas utilizadas en el artículo de NeRF. Para cada escena, deberá mostrar las curvas de entrenamiento de su modelo (*training loss* y PSNR en al menos una imagen de un conjunto de validación reservado). También deberá mostrar resultados cualitativos de cada método: muestre varios ejemplos de una imagen de referencia y su imagen predicha, tanto de vistas del conjunto de entrenamiento como de nuevas vistas del conjunto de prueba.
- **Entregen al menos una secuencia de imágenes** (video), donde hayan generado imágenes a lo largo de una trayectoria de cámara en movimiento continuo, similar a los ejemplos de la página del proyecto NeRF.

7. UdeSA 3D: *Structure-from-Motion*



En este trabajo podrán aplicar contenidos vistos en la primera mitad de la materia aplicados a la **estimación de movimiento libre de una cámara estéreo** en el espacio y la **reconstrucción 3D del entorno** observado.

La universidad de Maryland propone todos los años a sus estudiantes reconstruir una porción de la universidad **utilizando una cámara monocular**:

[Buildings built in minutes - An SfM Approach](#)

Esta es una descripción muy detallada de la solución que permite realizar una reconstrucción del entorno sin un objeto conocido en la escena.

En nuestro caso trabajaremos con cámaras estéreo las cuales permiten percibir distancias con un único par de imágenes haciendo más sencilla la metodología. Y además, simplificaremos el problema trabajando con secuencias de **imágenes tomadas contiguas en el tiempo**.

Pueden solicitar las cámaras estéreo de la cátedra para la realización del trabajo, teniendo que adaptar los algoritmos (de monocular a estéreo).

Odometría Visual utilizando cámaras estéreo

A continuación se enumeran de forma general el *pipeline* de estimación de la pose y reconstrucción 3D utilizando frames contiguos temporalmente. A esto se lo conoce como **Odometría Visual** (estimación de movimiento visual).

En una etapa previa al procesamiento de la secuencia de imágenes:

1. Calibración de la cámara estéreo:

Etapas de calibración para la obtención de los parámetros intrínsecos (distorsión, distancia focal, punto principal, etc..) y los parámetros

El primer frame estéreo (par de imágenes) de la secuencia a trabajar será utilizado para la inicialización del mapa y el eje de coordenadas global:

2. Inicialización del mapa 3D:

Deberá aplicarse el proceso de rectificación estéreo, extracción y matching de características entre imágenes, para luego triangularlas y así obtener un primer conjunto de puntos 3D (este proceso es análogo al aplicado para cada frame estéreo y será mejor detallado más adelante)

De esta manera, se considerará la **pose de la cámara izquierda del primer frame estéreo cómo el origen de coordenadas global**, es decir, el (0,0,0) del “mundo”.

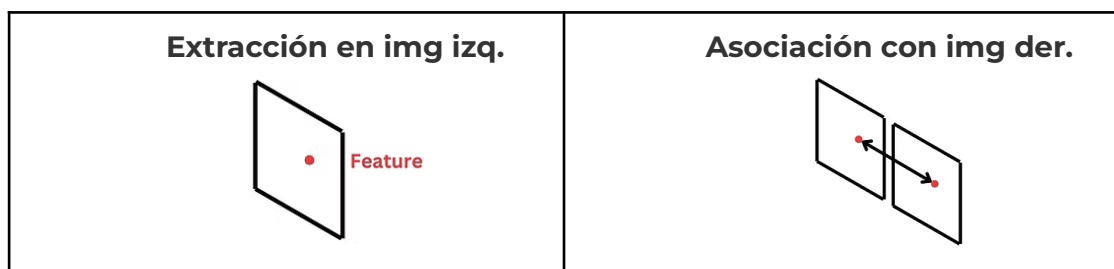
Luego, para todo frame estéreo (par de imágenes) de la secuencia se deberá realizar:

3. Rectificación de imágenes

Para cada frame estéreo (par de imágenes) deberán aplicar el proceso de rectificación estéreo utilizando los parámetros intrínsecos y extrínsecos.

4. Extracción y asociación de características visuales

Deberá extraer y asociar características visuales en ambas imágenes del frame estéreo.



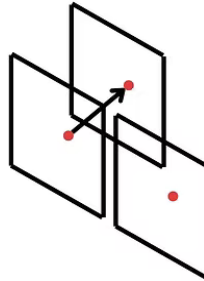
5. Asociar características visuales con el frame anterior

Deberán buscar asociaciones entre características visuales entre el frame estéreo actual y el frame estéreo anterior. Esto permite además saber qué características del frame actual corresponden a puntos 3D del mapa que ya han sido triangulados.

Para esto pueden utilizarse métodos de asociación de características ya trabajados cómo BFMatcher, FLANNMatcher o utilizar métodos de Flujo

Óptico ([Optical Flow](#)) que aprovechan el hecho de que los frame sean contiguos en el tiempo.

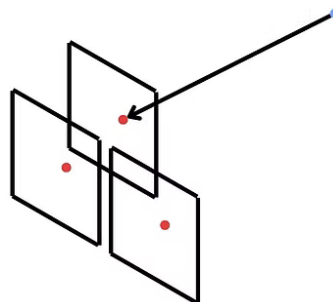
Asociación entre img izq. actual e img izq. anterior



6. Resolver la pose del frame estéreo actual

Se deberá utilizar métodos de Perspectiva por N Puntos (PnP) para resolver la pose del frame actual con respecto a los puntos del mapa 3D asociados (a través del frame anterior).

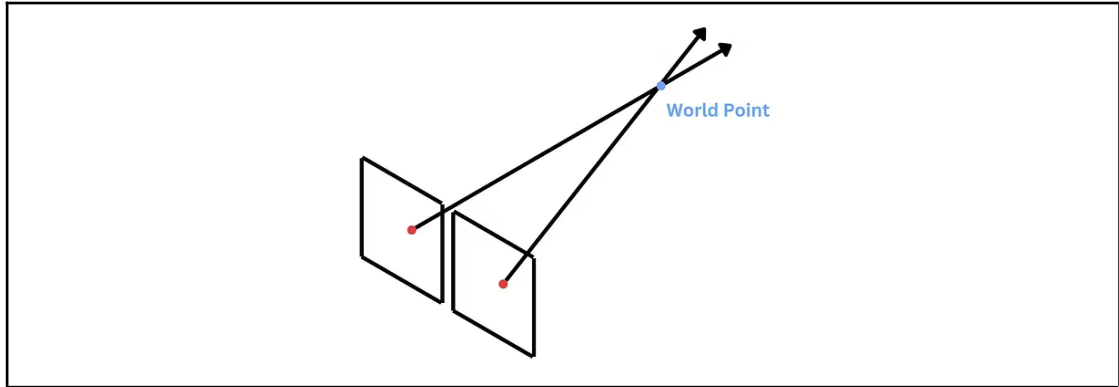
**Estimación de pose de la img izq. actual
utilizando la asociación con puntos 3D del mapa**



7. Triangulación de características visuales no asociadas

Toda característica visual extraída en el frame estéreo actual y que no se haya podido asociarse al frame anterior (y con el mapa) deberá ser triangulada y agregada al mapa 3D. Esto permite el crecimiento del mapa al explorar nuevas zonas del ambiente.

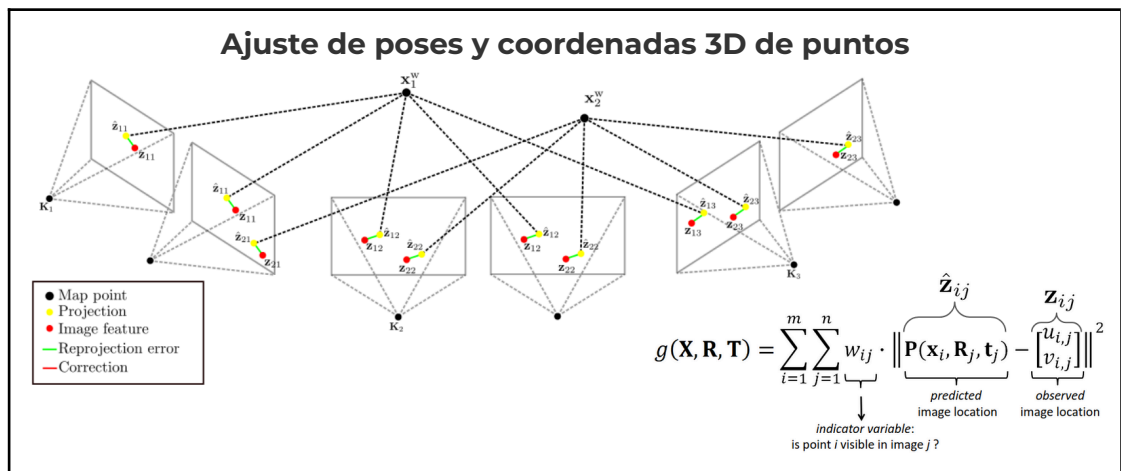
**Triangulación de características visuales estéreo
de elementos no mapeados**



8. Ajuste tanto de las poses de las cámaras como los puntos 3D del mapa

Se utiliza un método de optimización conocido como Bundle-Adjustment para ajustar todas las estimaciones de pose y coordenadas de puntos 3D del mapa.

Para esto es posible utilizar librerías eficientes como las disponibles en [scipy](#) ([large-scale BA in scipy](#)), la librería [pyceres](#) específicamente diseñada para este tipo de problemas o utilizar [COLMAP](#) con sus bindings de python [pyCOLMAP](#).



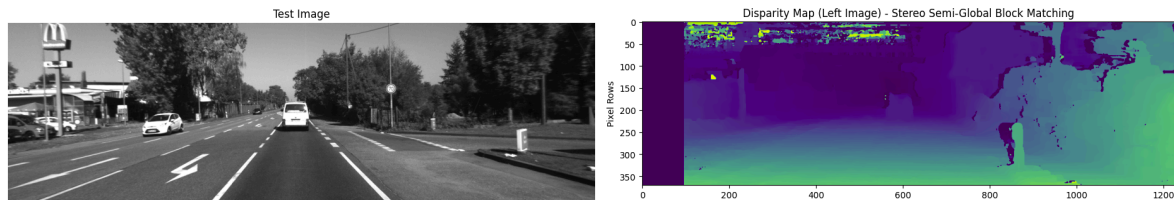
Esta serie de pasos debe ejecutarse iterativamente para todo frame estéreo de manera de obtener una secuencia de poses de cámara estimadas y puntos 3D del mapa ajustados.

Cómo paso final, luego de tener las poses de cámara y los puntos 3D bien ajustados es posible realizar la densificación del mapa 3D:

9. Cálculo de los mapas disparidad y triangulación del mapa de profundidad de todo frame estéreo

Utilizando los métodos ya vistos en la materia es posible densificar el mapa 3D obteniendo una reconstrucción densa “mucho más vistosa”.

KITTI Dataset



Cómo alternativa, pueden trabajar y probar el método en un subconjunto de imágenes del [KITTI Dataset](#).

Posee información de *ground-truth* de las poses de las cámaras utilizando mediciones GPS, lo cual permite comparar los resultados de las poses estimadas y además probar los métodos densificación de mapas utilizando las poses *ground-truth*.

Comparación con métodos del estado del arte

Se les pide comparar resultados con el sistema de SfM [COLMAP](#).

Opcionalmente podrían también aplicar métodos de reconstrucción basados en redes neuronales como el [SfMLearner](#) ([Unsupervised Learning of Depth and Ego-Motion from Video](#)) y [DeepSfM](#) (DeepSfM: Structure From Motion Via Deep Bundle Adjustment).

Teniendo una estimación de movimiento de toda la trayectoria luego es posible aplicar métodos modernos de reconstrucción 3D densos basados en *neural radiance fields* y *gaussian splatting*, [NerfStudio](#) y [GSplatStudio](#) (relacionado al enunciado de redes NeRFs).

En todos los casos deben dedicar espacio del informe a contar su funcionamiento y experiencia con ellos.

8. Propuesta de trabajo por estudiantes

Los grupos que se decidan por la opción de proponer un trabajo práctico no listado en los puntos anteriores, deberán proponer un problema a resolver que involucre los temas vistos en la materia.

Es obligatorio que el trabajo involucre la implementación de una arquitectura de redes neuronal. Pudiendo realizar un trabajo de análisis del estado del arte y la utilización de soluciones preexistentes en “modo inferencia”

Es un requisito que el tema sea revisado y aprobado previamente por el Profesor a cargo.

Para estructurar mejor el trabajo, se sugiere que el mismo siga el ciclo de desarrollo de un proyecto de inteligencia artificial el cual cuenta de los siguientes pasos:

1. Definición del problema a resolver
2. Estado del arte y trabajos relacionadas
3. Definición de un conjunto de datos
4. Creación del modelo
5. Evaluación del modelo y sus hiper parámetros
6. Despliegue y optimización

Tener especial cuidado en sobre-dimensionar la solución ni querer abarcar un problema demasiado grande. Lo mejor es un problema de tamaño moderado planificando una solución que les permita aprender de una determinada área o arquitectura.

1. Definición del problema a resolver

Para esto se deben plantear claramente los objetivos y metas a alcanzar durante el desarrollo del trabajo práctico. Además, se deben plantear hipótesis y proponer una metodología de trabajo que permita alcanzar los objetivos.

El problema a resolver tiene que poder definirse concretamente, debe ser acotado y plausible de implementar en menos del 30% del tiempo designado para la entrega.

2. Estado del arte y trabajos relacionadas

Es menester que el planteo del tema y las propuestas de resolución se basen en bibliografía y artículos publicada con referato, es decir, no cuentan como estado del arte páginas web Wikipedia, Medium, TowardsDatascience, ElRiconDelVago, Blogspot, posts en LinkedIn o redes sociales, etc. En estos últimos casos, se recomienda buscar siempre la referencia original y los artículos a dónde se hace mención.

Tener especial cuidado con ArXiv. En muchos casos, los artículos subidos a esta plataforma no fueron revisados todavía (no tienen referatos). Los autores de los trabajos suelen subir versiones preliminares de los artículos previas al envío a una revista o conferencia, en las cuales el trabajo podría ser rechazado, por lo que se recomienda hacer una búsqueda en internet para verificar el estado de publicación de los mismos.

Una fuente de trabajos para la exploración del estado del arte es Google Scholar: <https://scholar.google.com/>

3. Definición del conjunto de datos

Una de las tareas más importantes de este proyecto es la creación o definición del conjunto de datos de imágenes (o video) a utilizar. Prestar especial atención, durante la confección del estado del arte, a menciones sobre dataset públicos o sitios web donde los autores liberan los conjuntos de datos.

El dataset de imágenes a utilizar deberá estar estrechamente ligado al problema a resolver y contar con anotaciones que permitan extraer métricas de evaluación durante la evaluación y testeo del modelo.

Pueden tomar fotos ustedes mismos, buscar imágenes en internet, pensar en alguna otra fuente de imágenes como kaggle, huggingface, paperswithcode, etc. Tener en cuenta, en caso de utilizar un dataset propio o con anotaciones distintas a las necesarias, que el grupo deberá agregar manualmente las etiquetas deseadas en un número significativo de las mismas (este número dependerá del problema a resolver, consultar con el Profesor a cargo).

Luego de recolectar y etiquetar tu conjunto de datos, deberían dividirlo en conjuntos de entrenamiento, validación y prueba. Una regla general para dividir conjuntos de datos es asignar el 70% de tus datos a entrenamiento, 10% a validación y 20% a prueba.

Listas de datasets públicos disponibles:

- [Meta Pointer: A large collection organized by CV Datasets.](#)
- [Yet another Meta pointer](#)
- [ImageNet](#): a large-scale image dataset for visual recognition organized by [WordNet](#) hierarchy
- Vision-language datasets:
 - [Visual Genome](#)
 - [Flickr30k](#)
 - [VQA v2](#)
 - [ADE20K](#)
 - [LAION](#)
- [SA-1B](#): dataset of a large number of images and segmentation masks to segment objects in those images
- [COCO](#): large-scale object detection, segmentation, and captioning dataset
- [Open Images](#): a dataset of ~9M images annotated with image-level labels, object bounding boxes, object segmentation masks, visual relationships, and localized narratives
- [Cityscapes Dataset](#): This dataset focuses on semantic understanding of urban street scenes, with pixel-level annotations for various object classes such as cars, pedestrians, and roads
- [DeepFashion](#): a large-scale clothes dataset containing over 800,000 diverse fashion images annotated with bounding boxes, clothing categories, and attributes
- [Hugging face datasets](#): collection of generic datasets available on hugging face
- [Objaverse](#): a large-scale 3D asset database
- [SUN Database](#): a benchmark for scene recognition and object detection with annotated scene categories and segmented objects
- [Places Database](#): a scene-centric database with 205 scene categories and 2.5 millions of labelled images
- [NYU Depth Dataset v2](#): a RGB-D dataset of segmented indoor scenes
- [Microsoft COCO](#): a new benchmark for image recognition, segmentation and captioning
- [Flickr100M](#): 100 million creative commons Flickr images
- [Labeled Faces in the Wild](#): a dataset of 13,000 labeled face photographs
- [Human Pose Dataset](#): a benchmark for articulated human pose estimation
- [YouTube Faces DB](#): a face video dataset for unconstrained face recognition in videos
- [UCF101](#): an action recognition data set of realistic action videos with 101 action categories
- [HMDB-51](#): a large human motion dataset of 51 action classes
- [ActivityNet](#): A large-scale video dataset for human activity understanding
- [Moments in Time](#): A dataset of one million 3-second videos

4. Creación del modelo

El desarrollo del modelo se deberá realizar en Python y se recomienda utilizar la librería PyTorch. Otras librerías pueden utilizarse si el grupo de trabajo lo considera pertinente pero la cátedra no se compromete a brindar soporte sobre las mismas.

Se espera que el modelo sea entrenado sobre el conjunto de datos propuesto anteriormente. Pueden considerar tres maneras diferentes de entrenarlo:

- **Entrenar desde cero:** Inicializar el modelo aleatoriamente y entrenarlo desde cero en tu conjunto de entrenamiento.

- **Extracción de características:** Inicializar el modelo usando pesos pre-entrenados, extraer características del modelo y entrenar un modelo lineal sobre las características extraídas. Esto puede implementarse de varias maneras: podés usar PyTorch para agregar una nueva capa al modelo, congelar todas menos la última capa y entrenar en PyTorch; también podés extraer características del modelo y guardarlas en algún formato, luego entrenar un clasificador lineal con otro paquete como scikit-learn.
- **Ajuste fino (Fine-Tuning):** Inicializar el modelo usando pesos pre-entrenados en ImageNet; ajustar todo el modelo en tu conjunto de entrenamiento. Esto debería funcionar al menos tan bien como la extracción de características.

Es posible que necesites experimentar con diferentes optimizadores, tasas de aprendizaje, regularización y estrategias de aumento (augmentation) de datos

Se deberían mostrar las curvas de entrenamiento para los modelos, graficando la función de pérdida de entrenamiento por iteración, así como la precisión del modelo en los conjuntos de entrenamiento y validación en cada época.

5. Evaluación del modelo y sus hiper-parámetros

Analizar el rendimiento de los modelos propuesto mostrando ejemplos de clasificación o detección correcta/incorrecta, matrices de confusión y todo lo visto en clase. Discutir los resultados de los mejores modelos en el conjunto de pruebas y analizar cuán bien generalizan los mismos.

También se desea mostrar resultados cualitativos para analizar en detalle los casos buenos, malos y patológicos. Es necesario que propongan hipótesis o ideas de cómo se comportan modelos de manera analizando los resultados de forma visual.